
Optimizing Discrete Text Prompts with Reinforcement Learning

M.Alif Al Hakim¹ and Muhammad Mariozulfandy²

¹2206081250

²2206041404

Abstract

Language models can be adapted to specific tasks. Fine-tuning is a common adaptation method that updates all model parameters to achieve strong performance. However, this approach tightly binds the learned knowledge to the model and is computationally expensive. To address these limitations, prompt tuning has emerged as a more efficient alternative. This project reproduces and extends the RLPrompt method, which focuses on optimizing discrete text prompts to maintain interpretability and transferability, using reinforcement learning. We demonstrate the effectiveness of this approach on few-shot text classification tasks, utilizing DistilGPT2 as the policy model and DistilRoBERTa-base as the reward model. Our experiments show that RLPrompt achieves respectable results compared to supervised fine-tuning, while providing benefits in transferability, reduced GPU memory usage, and improved data efficiency. We also explore the difficulties of applying RLPrompt to an Indonesian dataset and discuss future research, including generating more human-readable prompts and refining the method for different languages.

1 Problem Definition

1.1 Prompt Tuning vs Fine Tuning

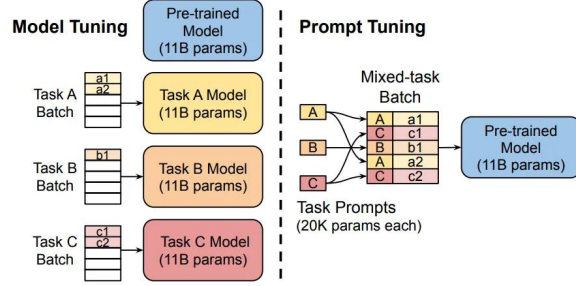


Figure 1: Comparison of Prompt Tuning and Fine Tuning (adapted from [Lester et al. \(2021\)](#)).

Fine tuning a pretrained model such as BERT or GPT-2 typically involves updating many or all of the model’s parameters using task-specific data. This comprehensive adjustment allows the model to integrate new, task-relevant knowledge deeply into its architecture, often leading to superior performance on the target task. However, this approach is computationally intensive, can require large amounts of data, and risks overfitting or causing the model to lose some of its general language understanding.

In contrast, prompt tuning offers a more lightweight alternative that leverages the model’s existing capabilities. Instead of modifying the model’s internal parameters, prompt tuning steers the model’s behavior by adding a specific piece of text, known as a prompt, to the input. This prompt is designed to guide the model toward producing desired outputs without the need for full retraining. While this method is significantly more efficient in terms of computational resources, it might not capture all the nuances required for a task compared to full fine tuning.

1.2 Soft Prompt Tuning

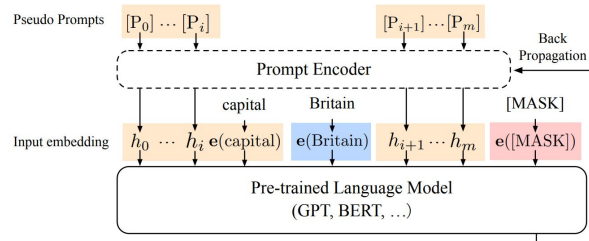


Figure 2: An example of soft prompt search for “The capital of Britain is [MASK]”. Given the context (blue zone, “Britain”) and target (red zone, “[MASK]”), the orange zone refer to the prompt (adapted from [Liu et al. \(2023\)](#)).

Soft tuning refers to the process of optimizing soft prompts—continuous embedding vectors—rather than discrete, human-readable tokens. These soft prompts are integrated into language models and adjusted through gradient descent, making them highly amenable to fine-tuning. Because they are continuous representations, they can be directly optimized by backpropagation during training, which often leads to more precise control over the model’s behavior compared to manually engineered discrete prompts.

However, this approach comes with notable drawbacks. One major issue is interpretability: since soft prompts are essentially numerical vectors, they are not easily understandable or interpretable by humans. This lack of transparency makes it challenging to diagnose errors or understand how changes in the prompt affect the model’s output. Moreover, soft prompts tend to be model-specific—they are tied to the internal architecture and embeddings of a particular language model, rendering them less transferable or compatible with other models. Additionally, many language models, especially those available only through inference APIs (like GPT-3), do not provide access to the necessary internal gradients required for optimizing soft prompts. This not only increases the computational cost but also restricts the applicability of soft tuning to scenarios where such gradients are readily available.

1.3 Discrete Prompt Tuning

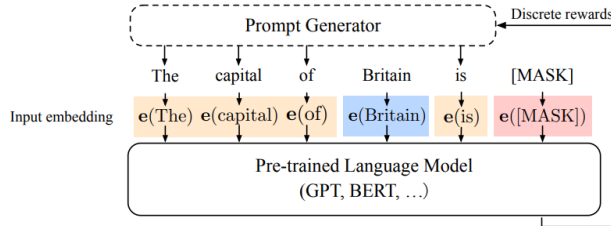


Figure 3: An example of discrete prompt search for “The capital of Britain is [MASK]”. Given the context (blue zone, “Britain”) and target (red zone, “[MASK]”), the orange zone refer to the prompt (adapted from [Liu et al. \(2023\)](#)).

Discrete prompt tuning is a technique for adapting large language models to specific tasks by finding and optimizing text-based prompts (actual words and phrases) rather than trainable embedding vectors. This approach doesn’t require access to the model’s internal mechanisms, produces human-readable prompts that can be understood and modified, and can be transferred between different models.

Discrete prompts are inherently challenging to optimize because they rely on fixed, human-readable text, and the space of all possible prompts is discrete rather than continuous. This means that small, incremental changes—common in gradient-based optimization—aren’t directly applicable.

As a result, discrete prompt tuning typically uses an “enumeration-then-selection” heuristic: you generate many candidate prompts (for example, by paraphrasing a base prompt) and then select the one that works best. However, this method does not systematically explore the entire prompt space, which can lead to suboptimal choices and missed opportunities for improvement. This situation creates a need for a method that can efficiently and automatically discover effective discrete prompts without requiring substantial manual effort or gradient-based fine-tuning of the large model itself.

2 Solution Design

2.1 Overview

This project aims to reproduce and extend the RLPrompt method [Deng et al. \(2022\)](#) for optimizing discrete text prompts using reinforcement learning (RL). The main idea is to automatically learn a fixed prompt—a sequence of T tokens chosen from the language model’s vocabulary—that, when combined with every input x , maximizes a task-specific performance metric R . This approach will be demonstrated on the task of **Few-Shot Text Classification**, by reformulating a classification problem as a masked language modeling task.

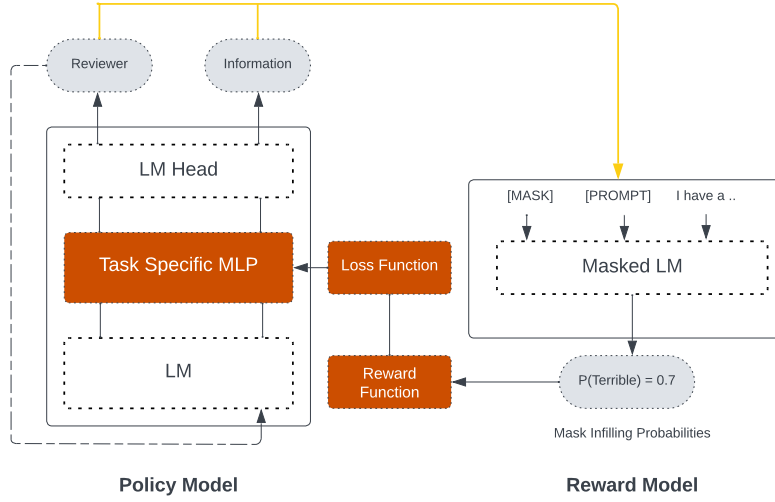


Figure 4: Overview of RLPROMPT for discrete prompt optimization. All LMs (white boxes) are frozen. We build our policy network by training a task-specific MLP module inserted into a frozen pre-trained LM. The figure above illustrates generation of a prompt (left), example usages in a masked LM for classification, and update of the MLP using RL reward signals (adapted from [Deng et al. \(2022\)](#)).

Additionally, we plan to explore this method on an Indonesian dataset and experiment with an alternative RL algorithm (e.g., Proximal Policy Optimization, PPO) to improve the stability and efficiency of reward optimization.

2.2 Markov Decision Process Model

In this project, the Markov Decision Process (MDP) is defined by the following components:

- **State Set ($\mathcal{S}$):** This set encompasses all possible sequences of tokens that can form a prompt of a fixed length T . Given a vocabulary size of $vocab_size$, the total number of possible states (prompts) is $(vocab_size)^T$. A state $s_t \in \mathcal{S}$ represents the specific prompt under consideration or active at timestep t .
- **Action Set ($\mathcal{A}$):** An action $a_t \in \mathcal{A}$ corresponds to a complete prompt, which is a sequence of T tokens generated by the Policy Model.
- **Timestep ($\mathcal{T} = \{1, 2, 3, \dots, t\}$):** This refers to each individual iteration in the process where the policy model generates a prompt.
- **State Transition Distribution (p):** This defines the probability of transitioning from the current prompt s_t to a new prompt s_{t+1} when the policy model takes action a_t (generating the prompt s_{t+1}). In this setup, if the action a_t is the generated prompt s_{t+1} , the transition is deterministic:

$$P(s_{t+1} | s_t, a_t) = 1$$

- **Initial State Distribution:** This describes the probability distribution over the starting states (prompts) at the beginning of the process (timestep $t = 0$).
- **(Immediate) Reward (R):** The details of the reward function are provided in Section 2.4.

2.3 Problem Formulation and Reinforcement Learning Setup

Discrete Prompt Optimization Problem

The goal is to find an optimal discrete prompt z^* (a sequence of T tokens) that maximizes the performance of a downstream task when used as part of the input to a frozen language model (LM). Formally, this is expressed as:

$$\max_{z \in V^T}; \frac{1}{|D|} \sum_{x \in D} R(y_{LM}(z, x)) \quad (1)$$

where:

- V is the LM’s vocabulary.
- $D = \{x_1, x_2, \dots, x_N\}$ is the dataset of task inputs,
- x is a task input from the dataset D .
- $y_{LM}(z, x)$ is the LM’s output when the prompt z is combined with an input x from the dataset D .
- $R(\cdot)$ is a reward function that measures task performance on a given output.

Reinforcement Learning Formulation

Because z is made of discrete tokens, we cannot use standard gradient descent. Instead, we cast the problem as a sequential decision-making process using RL. A policy network, parameterized by θ , generates the prompt token-by-token. This is formulated as:

$$\max_{\theta} \frac{1}{|D|} \sum_{x \in D} R(y_{LM}(\hat{z}, x)), \hat{z} \sim \prod_{i=1}^T \pi_{\theta}(z_i | z_{<i}, x) \quad (2)$$

where:

- z_i is the token chosen at step i .
- $z_{<i}$ represents all tokens generated before step i .
- $\pi_{\theta}(z_i | z_{<i}, x)$ is the policy network’s probability of choosing token z_i given the previous tokens and the input x .
- The objective is to adjust θ so that the average reward (obtained by the downstream LM) is maximized.

2.4 System Architecture and Training Process

Policy Network Architecture

- **Frozen LM Backbone:** We use a pre-trained language model (e.g., distilGPT-2) whose parameters remain fixed during training.
- **Trainable MLP (Policy Network):** A small multilayer perceptron (MLP) is inserted between LMs and its head. This MLP is the only trainable component and defines the policy π_{θ} .
 - **Input:** The contextual embeddings of already generated tokens $z_{<t}$ and input x .
 - **Output:** A probability distribution over the vocabulary for the next token.

During training, the policy network uses the current state to sample the next token from this probability distribution.

Training Steps

1. Prompt Generation:

For each training example (x, c) (where c is the class label for classification), the policy network generates a prompt $z = (z_1, z_2, \dots, z_T)$.

2. Input Combination and LM Output:

The generated prompt z is combined with the input x via a task-specific template, which in this case $[Mask][Prompt][Input]$. The combined input is then fed into the frozen LM to produce an output $y_{LM}(z, x)$.

3. Reward Computation:

The reward function R is computed on how well the LM predicts the correct label. One common approach uses a piecewise reward function:

$$R(x, c) = \lambda_1^{1-\text{Correct}} \cdot \lambda_2^{\text{Correct}} \cdot \text{Gap}_z(c) \quad (3)$$

where $\text{Gap}_z(c) := P_z(c) - \max_{c' \neq c} P_z(c')$, with $P_z(c) = \text{PLM}(c \mid z, x)$. Here, Correct is an indicator that is 1 if the prediction is correct, and λ_1, λ_2 are hyperparameters that scale the reward.

4. Reward Engineering and Stabilization:

The reward signals can be very noisy due to the complexity of the LM's output process. To stabilize training, two techniques are applied:

- **Input-Specific z-Score Normalization:**

For each input x , a batch of prompts $\mathcal{Z}(x)$ is sampled and their rewards $R_x(z)$ computed. The rewards are then normalized as:

$$\text{z-score}(z, x) = \frac{R_x(z) - \text{mean}_{z' \in \mathcal{Z}(x)} R_x(z')}{\text{stddev}_{z' \in \mathcal{Z}(x)} R_x(z')} \quad (4)$$

This normalization adjusts for differences in difficulty among inputs.

- **Piecewise Reward Function:**

To prevent the policy from exploiting a mis-specified reward function (e.g., by always choosing tokens that maximize one part of the reward), the reward is designed in a piecewise manner. This means combining a dense, quantitative signal (like the label probability) with a sparse, qualitative signal (like a bonus for achieving a certain accuracy threshold). This helps ensure the reward reflects the true task objective.

5. Policy Update:

Using a reinforcement learning method (e.g soft Q-learning), we update the parameters θ of the MLP to maximize the expected reward. The details about algorithm that we use and its loss calculation will be explained in section 3.3. The gradients are backpropagated through the MLP (via the frozen LM's contextual embeddings) to adjust θ .

6. Selection of Best Prompt:

Once training is complete, we evaluate candidate prompts on a validation set and choose the prompt $\hat{z}$ that yields the highest performance.

Inference Steps

After training, the best discrete prompt $\hat{z}$ (a fixed sequence of T tokens) is used for all new inputs. Given a new input x , we combine it with $\hat{z}$ using the same template as in training (e.g., $[\text{MASK}][\hat{z}][x]$). The frozen LM then predicts the token for the $[\text{MASK}]$ position using greedy decoding (i.e., always selecting the token with the highest probability). This predicted token is mapped to a class label via a verbalizer.

3 Solution Implementation

3.1 Model Architecture

In this project, we employ a distillGPT2 model as the prompt policy model, with an integrated Multi-Layer Perceptron (MLP) module inserted between its transformer layers. This MLP features a single hidden layer with 2048 units, and its input and output dimensions are both 768, matching the hidden state size of distillGPT2. During training, the weights of the distillGPT2 model are

frozen, allowing only the weights of the MLP model to be updated. For reward generation, we use distillRoberta-base for text classification. Figure 4 illustrates this architecture.

3.2 Training Process

Objective: The primary objective of our training process is to generate a sequence of T tokens to serve as input for the DistilRoBERTa-base classification model. During training, we utilize Boltzmann exploration that allows the policy model to probabilistically choose tokens based on their logits, rather than always selecting the token with the highest logit value. Specifically, tokens with higher logit values have a greater probability of being selected, encouraging exploration. Additionally, we employ teacher forcing algorithm, which leverages the ground truth token from the target model used in the reinforcement learning algorithm.

Prompt Construction: The process begins with the DistilGPT2 transformer module generating a hidden state. For the very first token in the sequence, the initial context is a special "end of token" (eot) marker. This generated hidden state is then passed through a task-specific Multi-Layer Perceptron (MLP) model. The output of this MLP is subsequently fed into the Language Model (LM) Head, which produces logits for the next token.

Algorithm 1 Generate Prompt in Training Mode

Initialize: Prompt $x_0 \leftarrow [\text{eot}]$
for $i = 1..T$ **do**
 $h_i \leftarrow \text{distillGPT2}(x_{i-1})$
 $h'_i \leftarrow \text{MLP}(h_i)$
 $\text{logits} \leftarrow \text{LMHead}(h'_i)$
 Sample a token z_i from logits
 $x_i \leftarrow z_i$
Output: Generated sequence $z = x_{1:T}$

Reward Computation Once a prompt or a sequence of T tokens is collected, it is fed into the respective reward model. The distillRoberta-base model receives the T tokens and generates logits for each token. To determine the predicted class, we identify a specific token acting as a "verbalizer." This verbalizer's logits are then passed through a softmax function to obtain class probabilities. The verbalizer serves as the decision-making token. For instance, in the SST-2 dataset, 'terrible' and 'great' might be used as verbalizers. If the probability (or logits before softmax) of 'great' is higher than 'terrible', the prediction is considered positive sentiment.

The resulting prediction and class probabilities are then used to compute the reward, as defined in Equation 3. The term $\text{Gap}_z(c)$ is calculated by subtracting the target class probability from the highest probability among the non-target classes. In this project, the incorrect coefficient (λ_1) is set to 180, and the correct coefficient (λ_2) is 200.

Task reward then will be transformed using Input-Specific z-Score Normalization (4)

3.3 Algorithms

The calculated reward is then utilized by the policy update algorithm. Reinforcement learning method is employed to update the parameters of the prompt policy model (DistillGPT2 with the integrated MLP). After the parameters of the task-specific MLP are updated based on the transformed reward, the training process proceeds to the next step.

Q-Learning (DQN)

Q-Learning is a value-based reinforcement learning method that learns the optimal action-value function $Q^*(s, a)$, representing the expected cumulative reward for taking action a in state s and following the optimal policy thereafter.

The **Bellman equation** for the optimal Q-function is:

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a') \mid s, a] \quad (5)$$

The **Q-learning update rule** is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (6)$$

For Deep Q-Networks (DQN), the **loss function** is:

$$L(\theta) = \mathbb{E}[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2] \quad (7)$$

where θ represents the neural network parameters, θ^- denotes the target network parameters, α is the learning rate, γ is the discount factor, and r is the immediate reward.

In the context of prompt policy optimization, the state s represents the current prompt configuration, action a corresponds to prompt modifications, and the reward r is derived from the calculated reward function for prompt effectiveness.

Soft Q-Learning (Soft-DQN)

Soft Q-Learning incorporates entropy regularization to encourage exploration and maintain policy diversity by maximizing both expected reward and policy entropy.

The **soft Bellman equation** is:

$$Q^*(s, a) = \mathbb{E}[r + \gamma(\max_{a'} Q^*(s', a') + \alpha \log \pi^*(a'|s')) \mid s, a] \quad (8)$$

The **soft Q-learning update** is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma(\max_{a'} Q(s', a') + \alpha \log \pi(a'|s')) - Q(s, a)] \quad (9)$$

The **soft policy** is computed as:

$$\pi(a|s) = \frac{\exp(Q(s, a)/\alpha)}{\sum_{a'} \exp(Q(s, a')/\alpha)} \quad (10)$$

The **temperature parameter update** is:

$$\alpha \leftarrow \alpha + \beta \nabla_{\alpha} \mathbb{E}[\alpha \log \pi(a|s) + \alpha \bar{H}] \quad (11)$$

where α is the temperature parameter controlling exploration, $\bar{H}$ is the target entropy, and β is the learning rate for temperature. The entropy regularization prevents premature convergence to suboptimal prompts and maintains exploration of the prompt space.

Proximal Policy Optimization (PPO)

PPO is a policy gradient method that directly optimizes the policy while preventing large policy updates that could destabilize training through a clipped objective function.

The standard **policy gradient objective** is:

$$L^{PG}(\theta) = \mathbb{E}[\log \pi_{\theta}(a|s) A(s, a)] \quad (12)$$

The **PPO clipped objective** is:

$$L^{CLIP}(\theta) = \mathbb{E}[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] \quad (13)$$

where the **probability ratio** is:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \quad (14)$$

The **value function loss** is:

$$L^{VF}(\theta) = \mathbb{E}[(V_{\theta}(s_t) - R_t)^2] \quad (15)$$

The **complete PPO objective** combines all components:

$$L^{PPO}(\theta) = \mathbb{E}[L^{CLIP}(\theta) - c_1 L^{VF}(\theta) + c_2 S[\pi_{\theta}](s_t)] \quad (16)$$

The **advantage function** is estimated using Generalized Advantage Estimation (GAE):

$$A_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \quad (17)$$

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (18)$$

where A_t is the advantage function, ϵ is the clipping parameter (typically 0.1-0.2), c_1 and c_2 are coefficients for value function and entropy terms, $S[\pi_\theta]$ is the entropy bonus term, R_t is the discounted return, γ is the discount factor, and λ is the GAE parameter.

For the DistillGPT2 with integrated MLP architecture, the **parameter update** follows:

$$\theta_{new} \leftarrow \theta_{old} + \alpha \nabla_{\theta} L^{PPO}(\theta) \quad (19)$$

where the MLP parameters are updated based on PPO gradients that incorporate the transformed reward signal from prompt effectiveness evaluation.

3.4 Evaluation and Testing

During evaluation or inference, to generate the sequence of T tokens, we employ a greedy decoding approach. At each step, we select the token with the highest logit score predicted by the Language Model (LM) Head as the next token in the sequence. This process continues until the desired length T is reached.

Following each evaluation cycle, we compare the accuracy (or other classification metrics) obtained for different generated prompts. The prompt that yields the highest performance is then selected as the best-performing prompt. This chosen prompt is subsequently inserted into a predefined template. The resulting filled template, now containing the optimized prompt, is then utilized for the intended downstream task.

4 Experiments

The Few-Shot Text Classification experiments utilized *DistilRoBERTa-base* as the reward model and *DistilGPT2* as the policy model. These experiments were conducted on SST-2 [Socher et al. \(2013\)](#), a binary classification dataset, and SST-5 [Socher et al. \(2013\)](#), a multi-class classification dataset. For SST-2, the verbalizers used were ['terrible', 'great']. For SST-5, the verbalizers employed were ['terrible', 'bad', 'okay', 'good', 'great']. All experiments were performed using NVIDIA P100 GPU.

Each few-shot experiment was trained using a 16-shot setting, meaning that for each label, there were 16 training examples and 16 validation examples. For testing, we used the entirety of the test set available for each dataset.

Each experiment involved a maximum of 6000 training steps, a learning rate of 5e-5, a batch size of 16, and a prompt length of 2 tokens. Each dataset was split into 5 distinct subsets, controlled by dataset seeds ranging from 0 to 4, with each seed defining its own training and evaluation set. To ensure reproducibility of the randomized training process, all experiments were conducted using a fixed random seed of 42.

For comparison, we also conducted experiments using a Supervised Fine-Tuning approach. These experiments used all available training and validation data from the original SST-2 and SST-5 datasets. To account for variability, we ran these fine-tuning experiments with 5 different random seeds, a batch size of 16, and for 2 epochs.

5 Results and Analysis

This section presents the results of various experiments conducted, along with detailed analysis.

5.1 RLPrompt

The following figures show the results of the RLPrompt experiments using each algorithm on the SST2 and SST5 datasets.

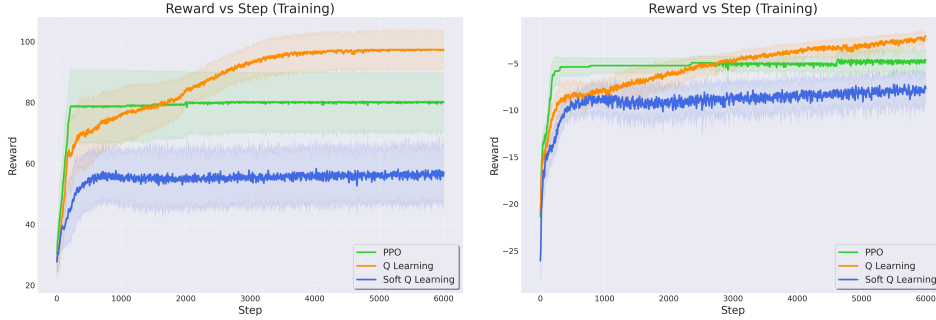


Figure 5: Training rewards vs. step for SST2 (left) and SST5 (right).

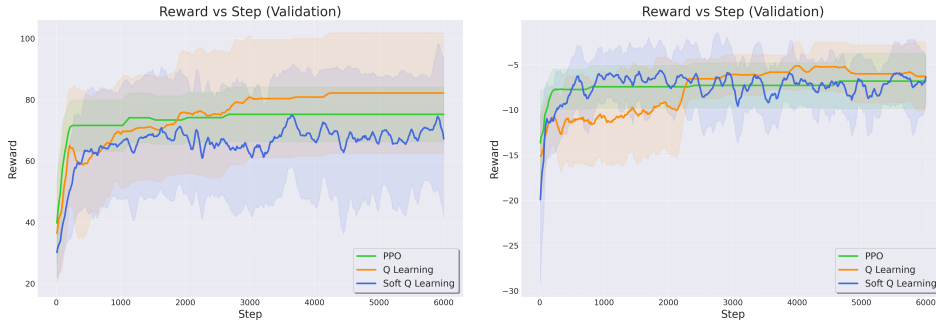


Figure 6: Validation rewards vs. step for SST2 (left) and SST5 (right).

The training phase is performed on the training dataset, where rewards or accuracies are measured at each step. The validation phase is executed every 10 steps to evaluate the policy model’s performance on unseen data. During validation, the generated prompts are paired with the validation inputs and fed to the language model (LM). Rewards and accuracies during validation are recorded without updating the policy model’s parameters. This allows us to assess generalization without affecting training.

The training dynamics show that PPO converged rapidly and stabilized early in the training process. Soft Q-Learning also converged quickly and stabilized, but showed more fluctuations and achieved lower training rewards compared to PPO. Q-Learning exhibited a more gradual learning process and ultimately achieved the highest training rewards among the three algorithms. In terms of validation performance, all algorithms performed similarly, with no algorithm showing a clear advantage based on accuracy or reward.

The reward reflects how effectively the reinforcement learning (RL) algorithm selects prompts to aid in the classification task, while accuracy measures the performance of the LM given the selected prompts. As training progresses, both reward and accuracy increase and eventually stabilize. The trend in accuracy closely follows the trend in reward, indicating that the RL algorithm consistently selects prompts that lead to improved LM classification performance.

5.2 Accuracy Comparison with Supervised Fine-Tuning

The RL algorithms demonstrated similar empirical performance, with PPO achieving slightly higher results by a very small margin. While Supervised Fine Tuning still outperforms RL-Prompt in terms of accuracy, the RL-Prompt methods nonetheless achieve respectable results. This suggests that although Fine Tuning remains the more accurate approach, RL-based methods offer competitive performance and may present advantages in scenarios where traditional supervision is less feasible.

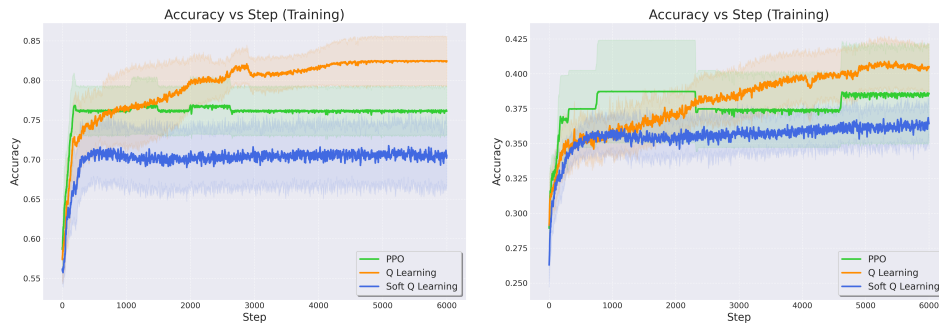


Figure 7: Training accuracy vs. step for SST2 (left) and SST5 (right).

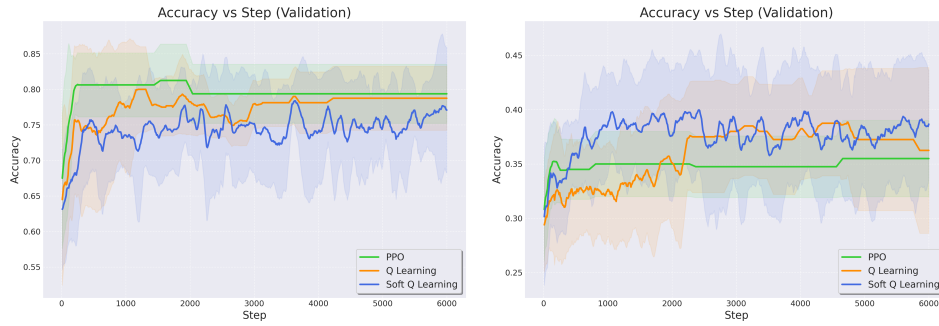


Figure 8: Validation accuracy vs. step for SST2 (left) and SST5 (right).

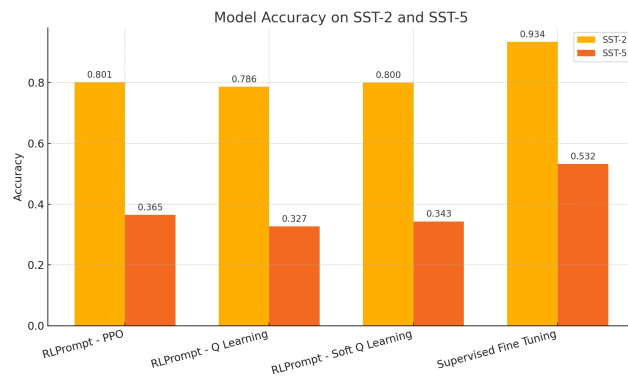


Figure 9: Average accuracy on test data for each RLPrompt algorithm and supervised fine tuning with distilroberta-base model

5.3 Transferability

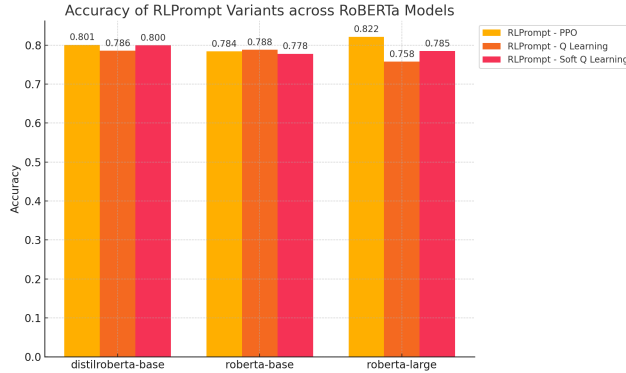


Figure 10: Evaluation results on test data using the best prompt obtained from training with distilroberta-base, applied to other models.

The best prompt obtained from training with distilroberta-base was subsequently tested on other models. All models demonstrated similar performance when using the prompt—comparable to that of distilroberta-base itself. This indicates that the prompts generated by RLPrompt are effectively transferable across different models, underscoring their generalizability and robustness.

5.4 GPU Usage

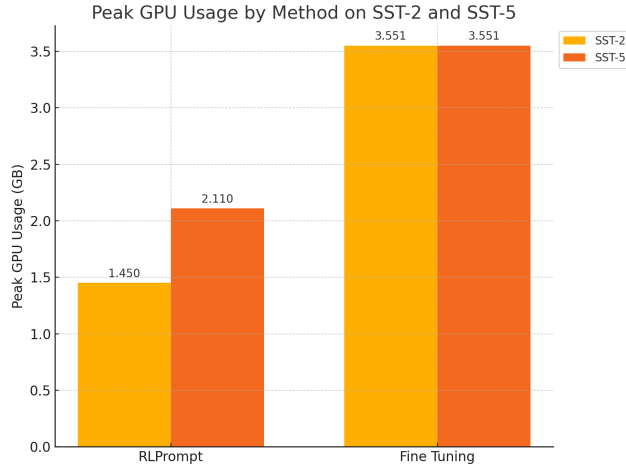


Figure 11: Peak GPU Usage of RLPrompt vs Supervised Fine-Tuning

RLPrompt operates by freezing the Language Model (LM) and updating only a small subset of its parameters, which leads to a substantial reduction in GPU usage compared to approaches that fine-tune the entire model. This efficiency makes RLPrompt a more resource-friendly alternative, enabling effective adaptation with lower computational costs.

6 Future Works

Gibberish Prompts Our experiments frequently identified that the best-performing prompts often sound like gibberish, lacking clear, human-understandable meaning. For example, the best prompts obtained for each dataset are shown on Table 1:

Table 1: Example of Best Prompts from Each Dataset

Dataset	Best Prompts
SST-2	Graphics equally, extremely extremely, Thus deeply
SST-5	uatesude, inessiness, Bot animation

This observation suggests that masked language models (like the reward model we employed) might utilize prompts in a way that differs from human comprehension. Given that fluent prompts are valuable for interpretability, future work could explore methods to generate more coherent prompts. Additionally, we can compare the performance of gibberish prompts and fluent prompts on the downstream task (classification, in this case) to understand how prompt fluency impacts model performance.

Experiments on Indonesian Dataset We also conducted experiments on an Indonesian dataset. However, the training process encountered some issues. As shown in Figure 12, the training rewards exhibit very minimal improvement, suggesting that the prompt policy model is unlearning. The rewards do not appear to be effectively guiding the policy model.



Figure 12: Learning Curve of RLPrompt with Indonesian Dataset

Additionally, based on Figure 13, the policy model appears to be trapped, consistently generating identical prompts during every validation step. This indicates that the policy parameters are not being updated efficiently, leading to an "entrapped" state.

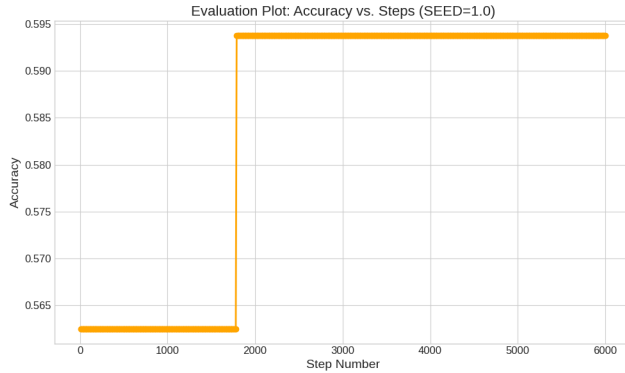


Figure 13: Validation Curve of RLPrompt with Indonesian Dataset

For future work, we could conduct further experiments to identify the root cause of this problem and ultimately find methods to successfully apply RLPrompt to Indonesian datasets. Potential avenues to explore include the defined template, the number of training steps, or even experimenting with cleaner datasets.

7 Conclusion

While Supervised Fine-Tuning currently demonstrates higher accuracy, RLPrompt presents itself as a compelling and promising alternative, offering several notable advantages. One key benefit of RLPrompt is its *transferability*. In contrast to traditional fine tuning or soft prompt methods, prompts learned through RLPrompt can be transferred across different models. This capability reduces the need for redundant training when switching between architectures. Furthermore, RLPrompt offers significant *efficiency* gains. It substantially lowers GPU memory usage, making it more accessible to users who may have less powerful hardware resources. Another advantage is its *low data requirement*. RLPrompt can achieve commendable performance even when trained with a limited amount of data. Finally, RLPrompt demonstrates *versatility* by integrating effectively with various reinforcement learning algorithms, including Q-Learning, Soft Q-Learning, and PPO.

These inherent characteristics underscore the considerable potential of RLPrompt for future advancements and applications in the realm of language model prompting.

References

- Deng, M., Wang, J., Hsieh, C.-P., Wang, Y., Guo, H., Shu, T., Song, M., Xing, E. P., and Hu, Z. (2022). Rlprompt: Optimizing discrete text prompts with reinforcement learning. *arXiv preprint arXiv:2205.12548*. (pages 3 and 4)
- Lester, B., Al-Rfou, R., and Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. *arXiv preprint arXiv:2104.08691*. (page 2)
- Liu, X., Zheng, Y., Du, Z., Ding, M., Qian, Y., Yang, Z., and Tang, J. (2023). Gpt understands, too. *arXiv preprint arXiv:2103.10385v2*. (pages 2 and 3)
- Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. Y., and Potts, C. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1631–1642. (page 9)